



Università degli Studi di Ferrara

UNIVERSITÀ DEGLI STUDI DI FERRARA
CORSO DI LAUREA IN INTELLIGENZA ARTIFICIALE

Compressione di immagini

Professore:

Prof. Giacomo DIMARCO

Studenti:

Dott. Enrico FAVALE

Dott. Alessandro DI TANNA

ANNO ACCADEMICO 2024 – 2025

Indice

	Page
1 Serie e Trasformata di Fourier	5
1.1 Serie di Fourier	5
1.2 Trasformata di Fourier	7
1.3 Trasformata discreta di Fourier (DFT)	9
1.4 Trasformata rapida di Fourier (FFT)	10
1.5 Trasformata di Fourier di dimensione due	11
2 FFT per compressione di immagini	13
2.1 Definizione della classe ImageCompressor	13
2.2 Compressione di un singolo canale	14
2.3 Compressione dell'immagine RGB	19
2.4 Utilizzo di un filtro alternativo	23
2.5 Ulteriori esempi di compressione	25
3 Le Wavelet	29
3.1 Motivazioni e limiti della trasformata di Fourier	29
3.2 Concetti fondamentali delle wavelet	32
3.3 Trasformata wavelet continua (CWT)	35
3.4 Trasformata wavelet discreta (DWT)	36

4	Wavelet per compressione di immagini	39
4.1	Compressione di un singolo canale	39
4.2	Compressione dell'immagine RGB	45
4.3	Esempio: Cubo di Rubik	48

Serie e Trasformata di Fourier

In questo capitolo vengono analizzate le principali evoluzioni teoriche e applicative della serie di Fourier, dalle sue origini fino agli strumenti computazionali attualmente utilizzati, come la trasformata rapida di Fourier (FFT). Dopo una panoramica sulla serie di Fourier e sulla sua capacità di rappresentare segnali periodici tramite combinazioni di seni e coseni, verrà introdotta la trasformata di Fourier continua, utile per segnali non periodici. Successivamente, si passerà alla formulazione discreta (DFT), fondamentale per l'elaborazione numerica dei segnali, per poi descrivere l'algoritmo FFT che consente di calcolare la DFT in maniera efficiente. Verrà trattata, infine, la trasformata bidimensionale, impiegata nella compressione e nel filtraggio delle immagini digitali.

1.1 Serie di Fourier

La rappresentazione delle funzioni tramite combinazioni di seni e coseni è una delle idee più potenti dell'analisi matematica e costituisce il cuore della teoria di Fourier. Introdotta da Joseph Fourier nei primi anni del 1800, questa teoria nasce come strumento per studiare la propagazione del calore, ma si è rapidamente affermata

come tecnica fondamentale in fisica, ingegneria e, più recentemente, in data science e intelligenza artificiale.

L'intuizione di Fourier è che una funzione periodica $f(x)$, regolare a tratti, può essere espressa come somma infinita di funzioni sinusoidali (seni e coseni) a frequenza crescente. Formalmente, una funzione $f(x)$ 2π -periodica può essere scritta come:

$$f(x) = \frac{a_0}{2} + \sum_{k=1}^{\infty} (a_k \cos(kx) + b_k \sin(kx))$$

dove i coefficienti a_k e b_k rappresentano l'"intensità" delle varie componenti in frequenza e sono calcolati mediante integrali:

$$a_k = \frac{1}{\pi} \int_{-\pi}^{\pi} f(x) \cos(kx) dx, \quad b_k = \frac{1}{\pi} \int_{-\pi}^{\pi} f(x) \sin(kx) dx$$

Questi coefficienti sono ottenuti tramite **proiezione ortogonale** della funzione $f(x)$ su una base di funzioni ortogonali: $\{\cos(kx), \sin(kx)\}$.

La Serie di Fourier può anche essere espressa in forma **complessa** utilizzando la formula di Eulero $e^{ikx} = \cos(kx) + i \sin(kx)$, ottenendo:

$$f(x) = \sum_{k=-\infty}^{\infty} c_k e^{ikx}$$

con i coefficienti complessi dati da:

$$c_k = \frac{1}{2\pi} \int_{-\pi}^{\pi} f(x) e^{-ikx} dx$$

Questa formulazione è particolarmente utile in ambito computazionale e teorico, poiché consente di semplificare i calcoli, introducendo una singola base esponenziale che sostituisce funzioni di seno e coseno separate. Inoltre, fornisce una simmetria tra le frequenze che aiuta a trattare similmente frequenze positive e negative, e rappresenta un ponte per introdurre la trasformata di Fourier.

Il **teorema di convergenza puntuale** implica che, data una funzione $f(x) : \mathbb{R} \rightarrow \mathbb{R}$ avente periodicità 2π , regolare a tratti ed esprimibile in somma di seni e coseni come:

$$S_N = \frac{a_0}{2} + \sum_{k=1}^N (a_k \cos(kx) + b_k \sin(kx))$$

Questa converge puntualmente, per $N \rightarrow \infty$ ad $f(x)$ nei punti in cui $f(x)$ è continua. Invece nei punti di discontinuità x_0 la serie di Fourier converge al valore medio dei limiti destro e sinistro:

$$\lim_{N \rightarrow \infty} S_N(x_0) = \frac{1}{2} (f(x_0^-) + f(x_0^+))$$

D'altra parte, la **disuguaglianza di Bessel** fornisce un limite superiore all'energia armonica, assicurando che la somma dei quadrati dei coefficienti a_k e b_k non superi l'energia totale del segnale:

$$\frac{a_0^2}{2} + \sum_{n=1}^{\infty} (a_n^2 + b_n^2) \leq \frac{1}{\pi} \int_{-\pi}^{\pi} |f(x)|^2 dx$$

Questo risultato è alla base della stabilità numerica in quanto implica la convergenza dei coefficienti di Fourier.

Quindi ricapitolando, i coefficienti a_k e b_k permettono di ricostruire $f(x)$ tramite la serie:

$$S_N(x) = \frac{a_0}{2} + \sum_{k=0}^N (a_k \cos(kx) + b_k \sin(kx))$$

Inoltre è possibile ignorare le armoniche ad alta frequenza, che spesso hanno un contenuto informativo minore, prendendo soltanto le prime N coppie. Questo meccanismo permette di eseguire *compressione* e *denoising*, eliminando le componenti a bassa energia.

Questo tipo di espansione è fondamentale nella pratica per approssimare funzioni complesse in maniera efficiente, ed è alla base di moltissime applicazioni moderne, dalla compressione audio/video all'analisi dei segnali EEG.

1.2 Trasformata di Fourier

La serie di Fourier rappresenta uno strumento efficace per semplificare funzioni periodiche, ma presenta dei limiti quando si lavora con segnali non periodici. In questi casi, la **trasformata di Fourier** ne costituisce una naturale estensione,

permettendo di analizzare funzioni su domini infiniti, senza ripetizioni. Si passa quindi da una somma discreta di seni e coseni (serie) a un intervallo **continuo** sulle frequenze.

L'idea fondamentale è quella di considerare il limite della serie di Fourier quando il dominio del periodo tende a diventare infinito. Se $f(x)$ è una funzione definita su $\mathbb{R}$ e sufficientemente regolare, la sua trasformata di Fourier è definita come:

$$\hat{f}(\omega) = \mathcal{F}(f(\omega)) = \int_{-\infty}^{\infty} f(x) e^{-i\omega x} dx$$

e la trasformata inversa è:

$$f(x) = \mathcal{F}^{-1}(\hat{f}(x)) = \frac{1}{2\pi} \int_{-\infty}^{\infty} \hat{f}(\omega) e^{i\omega x} d\omega$$

Queste due espressioni definiscono un **cambio di base** nello spazio $L^2(\mathbb{R})$, proiettando $f(x)$ su una base continua di esponenziali complessi $\{e^{i\omega x}\}$, che giocano lo stesso ruolo di $\{\sin(kx), \cos(kx)\}$ nella serie.

Proprietà fondamentali della trasformata di Fourier

- **Linearità:** Permette di scomporre in parti per poi trasformarle separatamente.

$$\mathcal{F}(\alpha f(x) + \beta g(x)) = \alpha \mathcal{F}(f) + \beta \mathcal{F}(g)$$

$$\mathcal{F}(\alpha \hat{f}(x) + \beta \hat{g}(x)) = \alpha \mathcal{F}^{-1}(\hat{f}) + \beta \mathcal{F}^{-1}(\hat{g})$$

- **Derivazione:** Risulta utile in quanto permette di semplificare equazioni differenziali parziali in equazioni differenziali ordinarie. Infatti se $f'(x)$ è derivabile e integrabile, allora:

$$\mathcal{F}\left(\frac{d}{dx} f(x)\right) = i\omega \mathcal{F}(f(x))$$

- **Teorema di Parseval:** L'energia totale di una funzione (ovvero l'integrale del suo quadrato) non cambia se la si rappresenta come somma di seni e coseni (cioè nel dominio di Fourier):.

$$2\pi \int_{-\infty}^{\infty} |f(x)|^2 dx = \int_{-\infty}^{\infty} |\hat{f}(\omega)|^2 d\omega$$

In altre parole la trasformata di Fourier preserva la norma L_2 a meno di una costante.

- **Convoluzione:** Definendo $f * g$ come convoluzione di due funzioni $f(x)$ e $g(x)$ si ha:

$$(f * g)(x) = \int_{-\infty}^{\infty} f(x - \xi)g(\xi) d\xi$$

In particolare nel caso della trasformata di Fourier si ha:

$$\mathcal{F}(f * g)(\omega) = \hat{f}(\omega)\hat{g}(\omega)$$

Risulta utile quindi calcolare $\hat{f}(\omega)$ e $\hat{g}(\omega)$ per poi moltiplicarli e calcolare la trasformata inversa $\mathcal{F}^{-1}$ in sostituzione ad una classica convoluzione che richiederebbe più tempo.

1.3 Trasformata discreta di Fourier (DFT)

Nella pratica, i segnali e le funzioni vengono elaborati dal computer sotto forma di sequenze finite di numeri. Per questo motivo, è necessario passare dalla trasformata continua alla sua controparte discreta, nota come **Trasformata Discreta di Fourier (DFT)**.

La DFT si applica a vettori finiti di N campioni $x_0, x_1, \dots, x_{N-1}$, tipicamente ottenuti campionando una funzione continua a intervalli regolari. In tal senso, la DFT rappresenta l'informazione usando funzioni seno e coseno con multipli interi della frequenza fondamentale $\omega_N = e^{-2\pi i/N}$.

La DFT è definita come:

$$\hat{f}[k] = \sum_{n=0}^{N-1} f[n] \cdot e^{-i2\pi kn/N}, \quad \text{per } k = 0, 1, \dots, N-1$$

dove:

- $f[n]$: valore del segnale al tempo n
- $\hat{f}[k]$: contenuto di frequenza alla frequenza k
- $e^{-i2\pi kn/N}$: funzione complessa esponenziale periodica

La trasformata inversa (IDFT) permette di tornare nel dominio del tempo:

$$f[n] = \frac{1}{N} \sum_{k=0}^{N-1} \hat{f}[k] \cdot e^{i2\pi kn/N}, \quad \text{per } n = 0, 1, \dots, N-1$$

La DFT può anche essere vista come un **prodotto tra una matrice e un vettore**:

$$[\hat{\mathbf{f}}[1\dots N]]^T = \mathbf{F} \cdot [\mathbf{f}[1\dots N]]^T$$

dove $\mathbf{F}$ è la matrice di Vandermonde $N \times N$ con elementi $\omega_N = e^{-i2\pi/N}$. La DFT quindi è una trasformazione lineare nello spazio complesso che produce un vettore $\hat{f}$ di elementi che possiedono sia modulo che fase (derivanti dai valori complessi della matrice di Vandermonde).

Applicazioni della DFT.

- Compressione audio e immagini (es. JPEG, MP3)
- Filtraggio di segnali
- Analisi spettrale (ricerca di frequenze dominanti)
- Riconoscimento di pattern periodici

Limiti della DFT. Un limite pratico della DFT è il suo costo computazionale: calcolare la DFT direttamente richiede $O(N^2)$ operazioni. Questo problema viene risolto grazie all'algoritmo della **Fast Fourier Transform (FFT)**, che verrà descritto nella prossima sezione.

1.4 Trasformata rapida di Fourier (FFT)

Come visto nel paragrafo precedente, il calcolo diretto della DFT richiede $O(N^2)$ operazioni, il che lo rende inefficiente per segnali o immagini di grandi dimensioni. Per superare questo limite, è stato sviluppato un algoritmo ottimizzato noto come **Fast Fourier Transform (FFT)**.

La FFT è un algoritmo che calcola la DFT in tempo $O(N \log N)$, riducendo drasticamente il numero di operazioni necessarie e rendendo la trasformata di Fourier utilizzabile in tempo reale e su dataset estesi.

Idea chiave

L'idea alla base della semplificazione della complessità sta nello sfruttare un approccio *divide et impera* separando il segnale $f[n]$ di lunghezza N in due sottosequenze di lunghezza $N/2$:

- $f_{\text{pari}}[n] = f[n] + f[n + N/2]$
- $f_{\text{dispari}}[n] = e^{(\frac{-i2\pi n}{N})}(f[n] - f[n - N/2])$

A questo punto per i k pari vale:

$$\hat{f}[2k] = \sum_{n=0}^{(N/2)-1} (f[n] + f[n + N/2])e^{(\frac{-i2\pi kn}{N/2})}$$

Per i k dispari vale:

$$\hat{f}[2k + 1] = \sum_{n=0}^{(N/2)-1} e^{(\frac{-i2\pi n}{N})}(f[n] - f[n + N/2])e^{(\frac{-i2\pi kn}{N/2})}$$

1.5 Trasformata di Fourier di dimensione due

La trasformata di Fourier bidimensionale (2D) è una naturale estensione della trasformata unidimensionale e viene comunemente utilizzata nell'elaborazione di immagini. Un'immagine digitale può infatti essere vista come una funzione $f(x_1, x_2)$ di due variabili spaziali, definita su una griglia rettangolare discreta.

Definizione continua 2D

Per una funzione continua $f(x_1, x_2) \in L^1(\mathbb{R}^2)$, la trasformata di Fourier bidimensionale è definita come:

$$\hat{f}(\omega_1, \omega_2) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x_1, x_2) \cdot e^{-i(\omega_1 x_1 + \omega_2 x_2)} dx_1 dx_2$$

La trasformata inversa è:

$$f(x_1, x_2) = \frac{1}{(2\pi)^2} \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} \hat{f}(\omega_1, \omega_2) \cdot e^{i(\omega_1 x_1 + \omega_2 x_2)} d\omega_1 d\omega_2$$

Trasformata discreta bidimensionale (2D DFT)

Nel contesto digitale, le immagini sono rappresentate da matrici $M \times N$ di pixel. Per una matrice $f[m, n]$, la DFT bidimensionale è:

$$\hat{f}[k, l] = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f[m, n] \cdot e^{-i2\pi\left(\frac{km}{M} + \frac{ln}{N}\right)}$$

E la trasformata inversa è:

$$f[m, n] = \frac{1}{MN} \sum_{k=0}^{M-1} \sum_{l=0}^{N-1} \hat{f}[k, l] \cdot e^{i2\pi\left(\frac{km}{M} + \frac{ln}{N}\right)}$$

In pratica, si applica la DFT:

- a ciascuna riga dell'immagine (trasformata orizzontale)
- poi a ciascuna colonna del risultato (trasformata verticale)

FFT per compressione di immagini

In questo capitolo viene descritto un approccio alla compressione di immagini basato sull'analisi in frequenza con la Trasformata di Fourier bidimensionale (FFT 2D). Verrà presentato il codice Python sviluppato per implementare tale tecnica, illustrando il comportamento del sistema dal punto di vista teorico e pratico.

2.1 Definizione della classe ImageCompressor

Il codice è organizzato in una classe Python chiamata `ImageCompressor` in Figura 2.1, che si occupa del caricamento dell'immagine, della compressione FFT su ciascun canale e della ricostruzione dell'immagine compressa. È prevista inoltre la possibilità di salvare gli spettri intermedi per fini didattici e di analisi. L'inizializzazione della classe avviene tramite il costruttore: `ImageCompressor(input_dir, image_name)` dove:

- `input_dir`: directory contenente l'immagine.
- `image_name`: nome del file immagine.

```
1 class ImageCompressor:
2     def __init__(self, input_dir, image_name):
3         self.input_dir = input_dir
4         self.image_name = image_name
5         self.image_path = f"{input_dir}/{image_name}"
6
7     def __compress_fft_channel(...):
8         ...
9
10    def compress_color_image_fft(...):
11        ...
12
```

Figura 2.1: Definizione della classe ImageCompressor

2.2 Compressione di un singolo canale

La funzione privata `__compress_fft_channel`, definita in Figura 2.2, è il cuore dell'algoritmo in quanto applica la compressione FFT a un singolo canale dell'immagine. I principali parametri che definiscono la funzione sono:

- `channel`: il canale su cui lavorare.
- `keep_fraction`: la percentuale di frequenze da tenere durante la compressione.
- `filter`: il tipo di filtro da utilizzare (il concetto verrà espanso più avanti).

Utilizzo della funzione `fft2` e `fftshift`

Il funzionamento è basato sull'esecuzione delle due fasi preliminari di `fft2` e `fftshift` riportate in Figura 2.3. La chiamata di queste due funzioni è centrale nel processo di compressione in quanto permette di “tradurre” il canale in uno spettro di frequenze e successivamente di riorganizzarle in maniera ottimale. In particolare:

- `fft2`: applica la trasformata di Fourier bidimensionale al canale iterando sulle colonne e sulle righe, convertendo i dati dallo spazio immagine (domi-

```
1 def __compress_fft_channel(  
2     self,  
3     channel,  
4     keep_fraction=0.1,  
5     filter="low-pass",  
6     save_spectrum=False,  
7     channel_name="",  
8     output_dir="spectra_output"  
9 ):  
10
```

Figura 2.2: Definizione della funzione `__compress_fft_channel`

```
1 fft = np.fft.fft2(channel)  
2 fft_shifted = np.fft.fftshift(fft)  
3
```

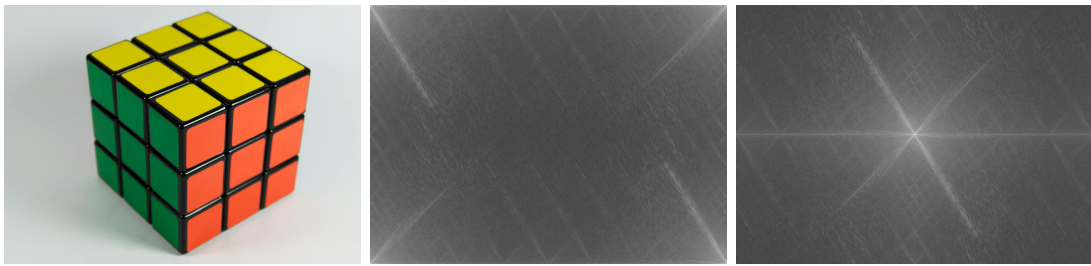
Figura 2.3: Chiamata delle funzione `fft2` e `fftshift`

nio spaziale) allo spazio delle frequenze. Ogni valore complesso risultante rappresenta un'ampiezza e fase per una particolare frequenza spaziale.

- **fftshift**: riposiziona le frequenze basse al centro dello spettro, rendendo più intuitiva l'analisi e la manipolazione. Senza lo shift, lo spettro avrebbe le frequenze basse agli angoli dell'immagine.

La Figura 2.4 mostra l'evoluzione dello spettro delle frequenze dell'immagine di un cubo di Rubik utilizzata come esempio. Partendo dall'input iniziale si può notare come prevalgano delle linee particolarmente marcate a delineare i dettagli dell'immagine. In questo senso è quindi prevedibile che nello spettro delle frequenze si vadano a riscontrare un mix di frequenze definite ed ordinate che rappresentano la struttura regolare e ripetitiva dell'immagine, in particolare i bordi netti tra i tasselli colorati.

L'applicazione della trasformata `fft2` consente di tradurre l'informazione spaziale in dominio di frequenza: nella rappresentazione risultante, le frequenze più basse sono localizzate negli angoli dell'immagine, mentre quelle più alte si distribuiscono attorno. Tuttavia, questa disposizione non è intuitiva per l'osservazione uma-



(a) Immagine originale (b) Spettro dopo `fft2` (c) Spettro dopo `fftshift`

Figura 2.4: Confronto tra immagine originale e spettro delle frequenze nei due stadi: `post-fft2` e `post-fftshift`. Si osserva come lo `fftshift` centri le frequenze basse. Gli spettri delle frequenze riportati sono relativi al canale “Blu” (B) dell’immagine.

na. Per questo motivo si applica successivamente la funzione `fftshift`, che ricentra lo spettro portando le componenti a bassa frequenza nel centro dell’immagine e spostando verso i bordi le alte frequenze.

Nel risultato finale dopo lo `fftshift`, si osserva una concentrazione luminosa al centro dell’immagine che indica la presenza predominante di frequenze basse, coerente con l’area uniforme dell’immagine, mentre le componenti ad alta frequenza, responsabili dei bordi e dei dettagli, sono relegate alle zone periferiche. Questa trasformazione è fondamentale per poter poi applicare un filtro spaziale selettivo (passa-basso o passa-alto) in maniera efficace.

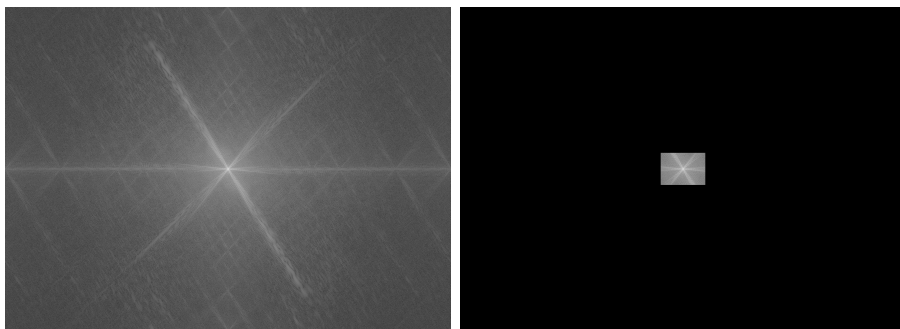
Creazione ed applicazione di una maschera

Il processo di compressione procede quindi lavorando direttamente con i valori delle frequenze trovate nel dominio trasformato. Come mostrato in Figura 2.5, viene costruita una maschera quadrata binaria centrata sul canale trasformato, con dimensioni proporzionali al parametro `keep_fraction`. Tale maschera ha lo scopo di selezionare le frequenze da mantenere, azzerando tutte le altre.

Nel caso mostrato, si applica un filtro passa-basso, mantenendo le frequenze centrali (quelle più basse) e scartando le alte frequenze ai margini. Questo tipo di filtro tende a rimuovere i dettagli fini, lasciando intatta la struttura globale del canale.

```
1 rows, cols = channel.shape
2 crow, ccol = rows // 2, cols // 2
3 r_keep = int(rows * keep_fraction / 2)
4 c_keep = int(cols * keep_fraction / 2)
5
6 mask = np.zeros_like(channel, dtype=bool)
7 mask[crow - r_keep : crow + r_keep, ccol - c_keep : ccol + c_keep]
   = True
8
9 fft_shifted_filtered = fft_shifted.copy()
10 fft_shifted_filtered[~mask] = 0
11
```

Figura 2.5: Creazione ed applicazione di una maschera allo spettro delle frequenze shiftato.



(a) Spettro prima dell'applicazione della maschera. (b) Spettro dopo l'applicazione della maschera.

Figura 2.6: Confronto tra lo spettro delle frequenze prima e dopo l'applicazione della maschera binaria per annullare le frequenze non importanti e mantenere solo quelle centrali. Si noti che la maschera applicata viene inizializzata impostando `keep_fraction` a 0.1. Gli spettri delle frequenze riportati sono relativi al canale “Blu” (B) dell'immagine.

```
1 fft_inverse = np.fft.ifft2(np.fft.ifftshift(fft_shifted_filtered))
2 return np.abs(fft_inverse)
3
```

Figura 2.7: Utilizzo dello shift inverso `ifftshift` e della trasformata inversa `ifft2` per ritornare ad un canale utilizzabile per ricostruire l'immagine compressa.

L'array `fft_shifted_filtered` contiene il nuovo spettro delle frequenze “compresso”, in cui le componenti ritenute meno significative sono state eliminate.

Visivamente è possibile rappresentare l'utilizzo della maschera creata come in Figura 2.6. Si noti come solo i valori al centro dello spettro vengono mantenuti mentre tutti gli altri vengono annullati. In tal senso, quindi, solo le frequenze più basse, contenenti le informazioni di maggiore importanza, vengono salvate.

Trasformata inversa

Dopo aver selezionato e mantenuto soltanto una parte delle frequenze tramite mascheratura, è necessario ritornare al dominio spaziale per ottenere un canale utilizzabile. A tal fine si utilizza la trasformata inversa di Fourier bidimensionale, in combinazione con lo shift inverso delle frequenze.

Come mostrato in Figura 2.7, le operazioni eseguite sono:

- **`ifftshift`**: ripristina la disposizione originale dello spettro, spostando il centro delle frequenze (ottenuto con `fftshift`) di nuovo ai bordi. Questo è necessario poiché la funzione `ifft2` si aspetta l'input non centrato per calcolare correttamente la trasformata inversa.
- **`ifft2`**: calcola la trasformata inversa di Fourier 2D, ricostruendo il canale dal suo spettro di frequenze filtrato.
- **`np.abs`**: restituisce il modulo dei numeri complessi risultanti dalla trasformata inversa. Questo è fondamentale perché il risultato di `ifft2` è in genere complesso (contenente parte immaginaria dovuta a errori numerici), mentre l'immagine finale deve essere rappresentata da valori reali (intensità dei pixel).

```
1 def compress_color_image_fft(  
2     self,  
3     keep_fraction=0.1,  
4     filter="low-pass",  
5     show=True,  
6     save_name="compressed.jpg",  
7     save_spectra=True  
8 )  
9
```

Figura 2.8: Definizione della funzione `compress_color_image_fft`.

2.3 Compressione dell'immagine RGB

La compressione totale dell'immagine avviene tramite chiamata alla funzione `compress_color_image_fft`, definita come in Figura 2.8, la quale ripete il processo di chiamata alla funzione `__compress_fft_channel`, descritta in precedenza, per ogni canale dell'immagine. In particolare, i canali utilizzati in questo esempio sono tre:

- R: rosso.
- G: verde.
- B: blu.

Per la modulazione dell'algoritmo risulta semplice modificare i canali utilizzati in base alle necessità. Per esempio si può adottare una scala di grigi (un canale) oppure CMYK (quattro canali).

Fase preliminare

La prima fase per la compressione dell'immagine si compone di alcune operazioni preliminari come mostrato in Figura 2.9.

Il primo passaggio del processo di compressione consiste nel caricamento dell'immagine a partire dal percorso fornito in fase di inizializzazione della classe.

```
1 img = Image.open(self.image_path).convert("RGB")
2 img_np = np.array(img)
3
```

Figura 2.9: Caricamento dell'immagine da file, conversione in formato RGB e trasformazione in array NumPy per consentire l'accesso diretto ai valori numerici dei singoli canali (Rosso, Verde, Blu) e permettere l'elaborazione frequenziale.

Questo avviene attraverso l'utilizzo della libreria PIL, che permette di aprire file immagine in diversi formati.

Successivamente, l'immagine viene convertita nel formato RGB, assicurando che essa sia composta esattamente da tre canali (Rosso, Verde e Blu), indipendentemente dal formato di partenza (grayscale, RGB, ecc.). Questa standardizzazione è fondamentale per garantire l'uniformità dell'elaborazione nei passaggi successivi.

Infine, l'immagine viene convertita in un array NumPy tridimensionale, dove ogni elemento rappresenta il valore d'intensità di un pixel per ciascun canale. Questa rappresentazione numerica consente di eseguire trasformazioni matematiche avanzate, come la trasformata di Fourier, e di manipolare direttamente le frequenze associate a ciascun canale dell'immagine.

Ciclo di chiamata alla funzione di compressione del canale

Una volta che l'immagine è stata convertita in un array NumPy tridimensionale, si accede ai tre canali di colore (Rosso, Verde e Blu) tramite indicizzazione lungo la terza dimensione dell'array: `img_np[:, :, i]` dove `i` varia da 0 a 2.

Per ciascun canale, viene invocata la funzione `__compress_fft_channel`, la quale applica il processo di compressione descritto nei paragrafi precedenti. Questa funzione restituisce una rappresentazione compressa del canale basata su una riduzione delle frequenze contenute nell'immagine secondo il parametro `keep_fraction` e la tipologia di filtro (che verrà analizzata nei paragrafi successivi).

Successivamente, l'array risultante viene sottoposto a due operazioni fondamentali:

- `np.clip(compressed, 0, 255)`: questa funzione garantisce che tutti i valori dei pixel compressi siano contenuti nel range valido per immagini a 8 bit.


```
1 for i in range(3):
2     channel = img_np[:, :, i]
3     compressed = self.__compress_fft_channel(
4         channel=channel,
5         keep_fraction=keep_fraction,
6         filter=filter,
7         ...
8     )
9     compressed = np.clip(compressed, 0, 255)
10    compressed_channels.append(compressed.astype(np.uint8))
11
```

Figura 2.10: Iterazione sui tre canali dell'immagine RGB. Per ciascun canale (Rosso, Verde, Blu) viene applicata la compressione tramite FFT, seguita da una normalizzazione dei valori nei limiti validi (0-255) e dalla conversione finale in interi a 8 bit per la successiva ricostruzione dell'immagine.

Poiché la trasformata inversa può introdurre piccoli errori numerici o valori fuori dai limiti, il `clip` è essenziale per mantenere la validità dell'immagine risultante.

- `compressed.astype(np.uint8)`: converte i valori numerici in interi senza segno a 8 bit, il formato standard utilizzato per rappresentare le immagini in molti formati grafici. Questa conversione è necessaria per poter salvare e visualizzare correttamente l'immagine compressa.

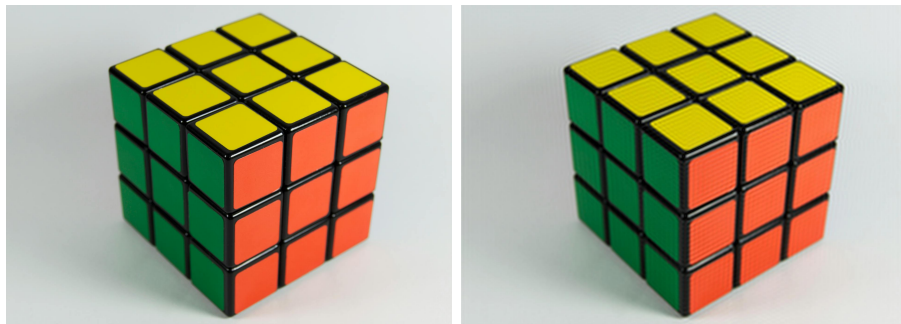
Infine, il canale compresso viene aggiunto alla lista `compressed_channels`, che conterrà i tre canali RGB pronti per essere ricombinati e ricostruire l'immagine finale.

Questo approccio modulare consente di trattare separatamente i canali dell'immagine e applicare strategie di compressione differenziate, se necessario, mantenendo al contempo una struttura di codice chiara e riutilizzabile.

Ricostruzione dell'immagine compressa

```
1 compressed_img = np.stack(compressed_channels, axis=2)
2 compressed_pil = Image.fromarray(compressed_img)
3
```

Figura 2.11: Ricostruzione dell'immagine compressa unendo i tre canali RGB processati singolarmente tramite `np.stack`, e conversione finale in formato immagine utilizzabile tramite `Image.fromarray`.



(a) Immagine originale del cubo di Rubik. (b) Immagine compressa del cubo di Rubik.

Figura 2.12: Confronto tra l'immagine originale del cubo di Rubik (a sinistra) e la sua versione compressa tramite FFT (a destra), con una frazione di frequenze mantenute pari al 10%. Si osserva come la compressione mantenga l'aspetto visivo generale pur riducendo la quantità di informazioni.

Una volta compressi separatamente i tre canali dell'immagine (rosso, verde e blu), il passo finale consiste nella loro ricombinazione per ottenere l'immagine compressa finale.

La funzione `np.stack(compressed_channels, axis=2)` consente di “impilare” i tre array bidimensionali (uno per ogni canale) lungo un terzo asse, ottenendo così un array tridimensionale con forma (altezza, larghezza, 3), ovvero il formato standard per un'immagine RGB.

Successivamente, l'array viene convertito in un oggetto immagine di tipo `PIL.Image` utilizzando la funzione `Image.fromarray(compressed_img)`. Questo oggetto può ora essere visualizzato, salvato su disco o ulteriormente analizzato.

Questa fase conclude il processo di compressione, restituendo un'immagine visivamente simile all'originale ma con una quantità inferiore di informazioni spettrali, e quindi con una potenziale riduzione dello spazio occupato o una maggiore efficienza nella trasmissione o memorizzazione. Per l'immagine trattata come esempio, il processo di conversione, con `keep_fraction` impostato a 0.1, ha prodotto risultati riportati in Figura 2.12.

2.4 Utilizzo di un filtro alternativo

Nel codice implementato, la funzione `__compress_fft_channel` permette di scegliere tra due tipi di filtro per la selezione delle frequenze da mantenere durante la compressione: un filtro passa-basso (“`low-pass`”) e un filtro passa-alto (“`high-pass`”).

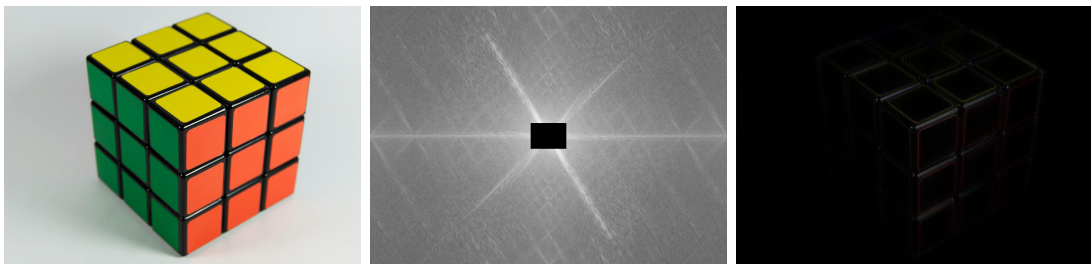
Come mostrato nel codice di Figura 2.5, la maschera viene costruita selezionando una porzione centrale dello spettro delle frequenze, corrispondente alle frequenze basse, che in genere rappresentano le componenti più significative e a bassa variazione spaziale dell'immagine. Applicando la maschera, le frequenze esterne vengono azzerate se si sceglie il filtro passa basso, mentre con il filtro passa alto vengono azzerate le frequenze centrali e mantenute quelle alte.

Questa distinzione viene gestita tramite il codice in Figura 2.13

La scelta tra filtro passa basso e passa alto influisce sul risultato visivo dell'immagine compressa, come pure sulla qualità e sulla dimensione del file risultante. Il filtro passa basso tende a preservare le componenti a bassa frequenza, spesso

```
1 if filter == "low-pass":  
2     fft_shifted_filtered[~mask] = 0  
3 else:  
4     fft_shifted_filtered[mask] = 0  
5
```

Figura 2.13: Definizione della condizione di applicazione del filtro “low-pass” o “high-pass”.



(a) Immagine originale del cubo di Rubik. (b) Spettro delle frequenze filtrato con filtro “high-pass”. (c) Immagine compressa con filtro “high-pass”.

Figura 2.14: Processo di compressione dell’immagine del cubo di Rubik con filtro “high-pass”. Si noti come lo spettro delle frequenze mantiene le frequenze più alte (esterne alla maschera centrale) per mettere in evidenza i dettagli dell’immagine e le sue feature basiche. Per semplicità si mostra solo lo spettro delle frequenze relativo al canale Blu (B) con `keep_fraction` a 0.1.

associate alle forme e ai contorni generali, mentre il filtro passa alto conserva principalmente i dettagli e le variazioni rapide di intensità, a scapito di una possibile perdita delle informazioni globali.

La flessibilità nel selezionare il tipo di filtro permette di esplorare diversi compromessi tra qualità visiva e compressione, rendendo l'algoritmo adattabile a differenti tipi di immagini e necessità di compressione.

Nell'esempio trattato nei capitoli precedenti, la modifica del filtro utilizzato produce un risultato completamente diverso che va a mettere in evidenza le feature basiche dell'immagine. Il processo e il risultato vengono mostrati in Figura 2.14.

2.5 Ulteriori esempi di compressione

Ponte di Brooklyn

Il primo esempio è un'immagine del ponte di Brooklyn di dimensione 2000 x 2000. Viene applicato filtro "low-pass" e filtro "high-pass" con `keep_fraction` a 0.1 in entrambi i casi. Il risultato dell'elaborazione del filtro "low-pass" è un'immagine che perde di qualità conseguentemente alla perdita del 90% delle frequenze più alte. Il risultato dell'elaborazione del filtro "high-pass" è un'immagine che mantiene solamente il 90% delle frequenze più alte ed annulla il 10% delle più basse mettendo in evidenza le feature più basilari dell'immagine.

Lupo

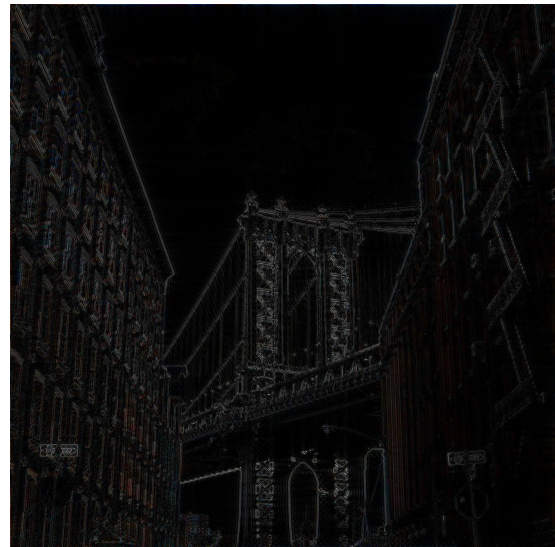
Il secondo esempio utilizza l'immagine di un lupo di dimensione 1920 x 1920. Anche in questo caso vengono applicati un filtro "low-pass" e un filtro "high-pass" con `keep_fraction` impostata, questa volta, a 0.05. Il filtro "low-pass" rimuove il 95% delle frequenze più alte, generando un'immagine comunque di alta qualità anche se conserva solo le componenti a bassa frequenza, come le aree uniformi. Il filtro "high-pass", al contrario, annulla il 5% delle frequenze più basse, mettendo in risalto i dettagli fini dell'immagine come contorni e trame del pelo anche se questi dettagli risultano appena visibili.



Figura 2.15: Immagine originale del ponte di Brooklyn.



(a) Risultato con filtro “low-pass”.



(b) Risultato con filtro “high-pass”.

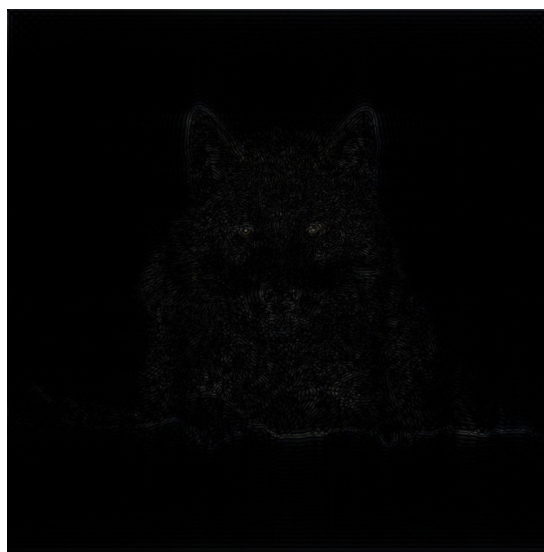
Figura 2.16: Risultati dell'applicazione dei filtri “low-pass” e “high-pass” all'immagine originale 2.15.



Figura 2.17: Immagine originale del lupo.



(a) Risultato con filtro “low-pass”.



(b) Risultato con filtro “high-pass”.

Figura 2.18: Risultati dell'applicazione dei filtri “low-pass” e “high-pass” all'immagine originale [2.17](#).

Le Wavelet

In questo capitolo viene introdotta la teoria delle wavelet come evoluzione naturale della trasformata di Fourier. Dopo aver discusso i limiti della trasformata di Fourier nella rappresentazione di segnali non stazionari, in particolare la perdita dell'informazione temporale, si presenta la teoria delle wavelet come soluzione per ottenere una rappresentazione tempo-frequenza localizzata. Vengono illustrate sia la trasformata wavelet continua (CWT), utile per l'analisi concettuale, sia la trasformata wavelet discreta (DWT), alla base delle applicazioni numeriche.

3.1 Motivazioni e limiti della trasformata di Fourier

L'analisi di Fourier ha rappresentato una svolta epocale nella comprensione e nel trattamento dei segnali. La sua capacità di scomporre una funzione in una somma (o integrale) di sinusoidi di frequenza crescente fornisce un quadro preciso della componente spettrale del segnale. Tuttavia, quando ci si confronta con segnali *non stazionari*, cioè con contenuto informativo che varia nel tempo (o nello spazio), la trasformata di Fourier mostra limiti strutturali. In questo contesto nasce l'esigenza

di strumenti alternativi in grado di rappresentare simultaneamente contenuti in frequenza e in tempo.

Perdita di localizzazione temporale

La trasformata di Fourier tradizionale scompone un segnale $f(t)$ come combinazione lineare di funzioni esponenziali complesse $e^{i\omega t}$ che oscillano indefinitamente su tutto il dominio. Questo comporta che ciascuna componente spettrale sia definita in modo *globale*, senza alcuna informazione sulla posizione temporale del segnale.

Esempio: consideriamo due segnali distinti:

- $f_1(t)$: una sinusoide a 440 Hz attiva per tutta la durata del segnale.
- $f_2(t)$: la stessa sinusoide ma attiva solo per un breve intervallo di tempo.

Entrambi avranno una componente spettrale a 440 Hz nella trasformata di Fourier, ma la trasformata non sarà in grado di distinguere *quando* quella frequenza è presente (come in Figura 3.2). In altre parole, la trasformata fornisce un'informazione completa sulle **frequenze presenti**, ma **ignora la loro collocazione temporale**.

Questa caratteristica è inadeguata per l'analisi di eventi localizzati nel tempo come impulsi, discontinuità, rumori transitori, picchi o mutamenti improvvisi di un segnale. Il concetto può essere esteso alle immagini non stazionarie che siano localmente o globalmente dinamiche.

Trasformata di Gabor

La tecnica presentata da Gabor si offre di risolvere il problema di localizzazione temporale e frequenziale della trasformata di Fourier. Difatti la trasformata di Gabor è una versione della **Short-Time Fourier Transform (STFT)**. La *STFT* permette di analizzare un segnale dividendolo in **finestre temporali** e applicando la trasformata di Fourier su ognuna di queste finestre, ottenendo una rappresentazione simultanea di **tempi e frequenze**.

$$G[f](t, \omega) = \int_{-\infty}^{\infty} f(\tau)g(\tau - t)e^{-i\omega\tau} d\tau$$

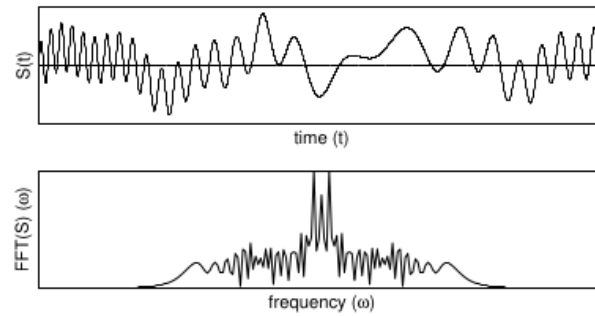


Figura 3.1: Il segnale $S(t)$ è chiaramente composto da varie componenti di frequenza che si manifestano in tempi diversi. Ad esempio, all'inizio del segnale, ci sono componenti ad alta frequenza. Al contrario, la parte centrale del segnale presenta oscillazioni a bassa frequenza. Se il segnale rappresentasse musica, allora l'inizio del segnale produrrebbe note alte, mentre la parte centrale produrrebbe note basse. La trasformata di Fourier del segnale contiene tutte queste informazioni, ma non c'è alcuna indicazione su quando le note alte o basse si verificano nel tempo. Infatti, per definizione, la trasformata di Fourier elimina tutte le informazioni del dominio del tempo.

Qui, $g(\tau - t)$ è una finestra temporale che localizza la trasformata a un certo intervallo di tempo t . La funzione $g(\tau - t)$ agisce come un filtro temporale che isola una parte del segnale a un dato tempo t , mentre $e^{-i\omega\tau}$ esegue la trasformata di Fourier su questa finestra.

La Trasformata di Gábor può quindi essere utilizzata per ottenere una rappresentazione del segnale nel dominio tempo-frequenza dove, t è il tempo e ω è la frequenza.

In pratica, si ottengono valori locali di frequenza per ogni istante di tempo.

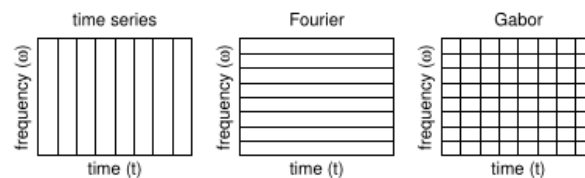


Figura 3.2: Grafico con le differenze tra un analisi di serie temporale, analisi di Fourier e analisi di Gábor per un segnale

La lunghezza della finestra risulta un punto cruciale in quanto non si può ottenere una risoluzione in tempo e in frequenza perfetta simultaneamente:

- Una finestra più **corta** fornisce una migliore risoluzione temporale, ma peggiora la risoluzione in frequenza, impedendo di catturare correttamente le frequenze basse.
- Una finestra più **lunga** offre una migliore risoluzione in frequenza, ma riduce la risoluzione temporale, rendendo più difficile localizzare cambiamenti rapidi nel segnale.

3.2 Concetti fondamentali delle wavelet

Il passo evolutivo successivo alla *STFT* sta nel consentire alla **dimensione della finestra** di **variare** progressivamente ottenendo una migliore risoluzione temporale. Inizialmente, vengono estratte le componenti a bassa frequenza, questo a causa della loro lentezza e delle loro lunghe oscillazioni. In seguito la finestra viene accorciata gradualmente per poter estrarre frequenze più alte. In quanto più rapide nei cambiamenti, queste frequenze, richiedono finestre più piccole per poter localizzare meglio l'istante in cui la componente è presente. In questa maniera, al termine dell'analisi, si avrà una risoluzione ottima sia in tempo che in frequenza. Questo è il principio fondamentale della *wavelet theory*. Il termine “*wavelet*” sta per piccola onda e deriva dal fatto che la finestra estrae porzioni di onda sempre più piccole.

Funzione madre e trasformazioni

Le wavelet utilizzano una *funzione madre* (o *wavelet madre*), indicata generalmente con $\psi(t)$, da cui si generano tutte le altre wavelet tramite operazioni di traslazione e scalatura.

$$\psi_{a,b}(t) = \frac{1}{\sqrt{|a|}} \psi\left(\frac{t-b}{a}\right)$$

a e b sono rispettivamente il parametro di scalatura e quello di traslazione.

Esistono diversi tipi di onde madre ognuna progettata per avere proprietà benefiche per un determinato problema.

L'esempio per eccellenza è la *wavelet di Haar* che gode di un supporto compatto che la rende adatta ad avere una localizzazione precisa nel dominio temporale. È infatti in grado di rilevare cambiamenti rapidi nel tempo come picchi o discontinuità, tuttavia, è meno precisa nel dominio delle frequenze.

Traslazioni e scale

Ogni wavelet deriva dalla funzione madre $\psi(t)$, modificata tramite due operazioni fondamentali:

- **Scala** (a): regola quanto la wavelet è “stirata” o “compressa”. Valori piccoli di a analizzano dettagli (alta frequenza), mentre valori grandi evidenziano tendenze generali (bassa frequenza).
- **Traslazione** (b): sposta la wavelet lungo l'asse del tempo, permettendo di analizzare diverse porzioni del segnale.

Nella versione discreta, DWT, si usano valori di scala e traslazione scelti in modo regolare, ad esempio $a = 2^j$ e $b = k \cdot 2^j$ con $j, k \in \mathbb{Z}$.

Ortogonalità

Molte famiglie di wavelet, come quelle di Haar, sono costruite in modo da essere ortogonali. Questo significa che le diverse wavelet sono **indipendenti** tra loro, non si sovrappongono nell'informazione che forniscono, contribuendo in maniera indipendente alla rappresentazione del segnale.

Una wavelet è ortogonale se la famiglia di funzioni $\psi_{m,n}$ è ortogonale. Ne consegue che un segnale o funzione dati può essere espressa in maniera univoca:

$$f(t) = \sum_{m,n} c_{m,n} \psi_{m,n}(t)$$

dove i coefficienti $c_{m,n}$ sono determinati dall'ortogonalità:

$$c_{m,n} = \langle f, \psi_{m,n} \rangle$$

Analisi Multi-Risoluzione

Il potere delle wavelet sta nella capacità di prendere una funzione segnale $f(t)$ ed esprimerla come limite di successive approssimazioni, ognuna delle quali è una versione sempre più fine della funzione nel tempo. Queste risoluzioni corrispondono a **diversi livelli di risoluzione**.

Un'analisi multi-risoluzione (**MRA**) è un metodo che fornisce un approccio formale per costruire il segnale con diversi livelli di risoluzione. Questo matematicamente si traduce in una sequenza di sottospazi incastonati di L^2 . In particolare:

- I sottospazi possono essere incastonati uno nell'altro.
- L'unione dei sottospazi copre tutto L^2 .
- L'intersezione risulta nulla.
- Ogni sottospazio acquisisce una diversa risoluzione.

Inoltre esiste una funzione ϕ

$$\phi_{0,n} = \phi(x - n)$$

chiamata scaling function o wavelet padre. Tale funzione viene messa in relazione con la wavelet madre tramite:

$$\psi(x) = \sqrt{2} \sum_{n=-\infty}^{\infty} (-1)^{n-1} \bar{c}_{-n-1} \phi(2x - n)$$

dove

$$c_n = \sqrt{2} \int_{-\infty}^{\infty} \phi(x) \bar{\phi}(2x - 1) dx$$

La funzione di *scaling* ϕ è utilizzata per costruire sottospazi a bassa risoluzione, che catturano le principali componenti a bassa frequenza. La *wavelet madre* ψ è utilizzata per i dettagli del segnale, cioè le componenti ad alta frequenza.

È importante sapere che data una wavelet ψ ed una funzione ψ limitata ed integrabile la convoluzione $\psi * \phi$ sarà anch'essa una wavelet. Questo è fondamentale per poter creare altre wavelet partendo da quella madre.

3.3 Trasformata wavelet continua (CWT)

La trasformata wavelet continua (CWT) permette di analizzare un segnale $f(t)$ in termini di frequenza e tempo. Questo avviene scalando e traslando la funzione madre $\psi(t)$ applicata al segnale. Ogni wavelet risultante è adattata per trattare il segnale a diverse risoluzioni temporali e spaziali con una rappresentazione **continua**. Si definisce la trasformata wavelet continua come:

$$\mathcal{W}_f(a, b) = (f, \psi_{a,b}) = \int_{-\infty}^{\infty} f(t) \bar{\psi}_{a,b}(t) dt$$

dove:

- $a \in \mathbb{R}^+$ è il parametro di scala,
- $b \in \mathbb{R}$ è la traslazione temporale,
- $\bar{\psi}$ indica il complesso coniugato della funzione madre.

Il risultato $\mathcal{W}_f(a, b)$ è una funzione bidimensionale che rappresenta il contenuto del segnale $f(t)$ alla scala a e alla posizione b .

In questo caso la wavelet madre funge da kernel della trasformazione, nel caso di Fourier il kernel era definito come $K(t, \omega) = e^{-i\omega t}$.

Il principio della trasformata wavelet è piuttosto semplice. In primo luogo, il segnale viene suddiviso in una serie di segnali più piccoli traslando la wavelet con il parametro b su tutto il dominio temporale del segnale. In secondo luogo, lo stesso segnale viene processato su diverse bande di frequenza, o risoluzioni, scalando la finestra della wavelet con il parametro a . La combinazione di **traslazione** e **scalatura** consente di elaborare i segnali a diversi tempi e frequenze.

Nella Figura 3.3 il riquadro che illustra la trasformata wavelet mostra il concetto di multi-risoluzione in azione. Partendo da una finestra del dominio di Fourier larga, viene estratto l'intero contenuto di frequenza. La finestra temporale viene quindi ridotta a metà, portando a una risoluzione temporale più fine a scapito di una peggiore risoluzione in frequenza. Questo processo continua fino a ottenere la risoluzione tempo-frequenza desiderata.

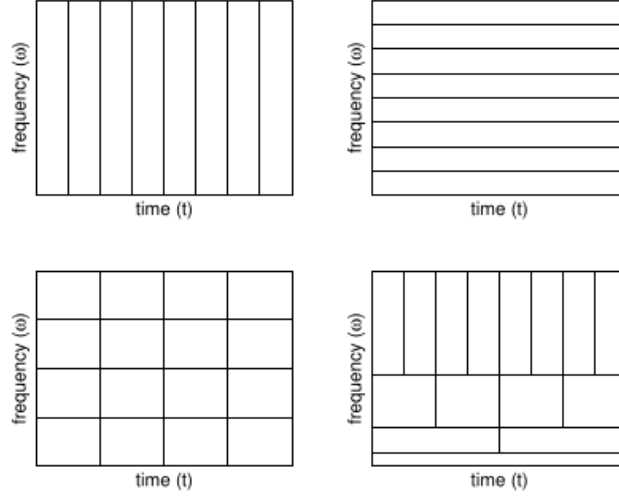


Figura 3.3: Grafico con le differenze tra un'analisi di serie temporale, analisi di Fourier, analisi di Gabor e analisi wavelet per un segnale.

3.4 Trasformata wavelet discreta (DWT)

La trasformata discreta prende in input dati, come un'immagine, che sono campionati in intervalli discreti. Utilizzando la trasformata wavelet discreta l'immagine viene decomposta in vari livelli di risoluzione, e in seguito una versione compressa viene ricostruita mantenendo solo i livelli di risoluzione desiderati. Mantenendo meno livelli di risoluzione, la qualità dell'immagine può essere compromessa, ma si ottiene una buona compressione. La tecnica risulta utile per ridurre le dimensioni dell'immagine.

La *DWT* è definita in termini di scalatura e traslazione di una wavelet madre in base ad un *lattice*. Questo rappresenta una griglia discreta di valori di tempo e spazi su cui viene applicata la trasformata:

$$\psi_{m,n}(x) = a_0^{-m/2} \psi(a_0^{-m}x - nb_0), \quad \text{con } a_0, b_0 > 0 \text{ e } m, n \text{ interi}$$

Quello che avviene è proprio la discretizzazione dei valori di traslazioni e scalature. Si definisce quindi la trasformata discreta:

$$\mathcal{W}_\psi(f)(m, n) = \int_{-\infty}^{\infty} f(t) \bar{\psi}_{m,n}(t) dt = a_0^{-m/2} \int_{-\infty}^{\infty} f(t) \bar{\psi}(a_0^{-m}(t) - nb_0) dt$$

Inoltre se $\psi_{m,n}$ è completa, il segnale o la funzione può essere espansa nelle wavelet base:

$$f(t) = \sum_{m,n=-\infty}^{\infty} (f, \psi_{m,n}) \psi_{m,n}(t)$$

I parametri di scalatura e dilatazione sono tipicamente scelti come $a_0 = 2$ $b_0 = 1$ corrispondenti a dilatazioni 2^{-m} e traslazioni di $n2^m$.

Il lattice consente di ottenere una rappresentazione sparsa dei segnali, dove solo una piccola parte delle wavelet (quelle non nulle) contribuisce effettivamente alla ricostruzione del segnale o immagine. Le wavelet vengono infatti usate dall'algoritmo *JPEG2000* la cui idea è quella di conservare solo il necessario per ricostruire l'immagine con il livello di accuratezza desiderato.

Wavelet per compressione di immagini

In questo capitolo viene analizzato il processo di compressione di immagini digitali mediante trasformata wavelet discreta (DWT). A differenza di approcci tradizionali come la FFT, le wavelet permettono una rappresentazione gerarchica e multi-risoluzione del segnale visivo, risultando particolarmente efficaci nel conservare le informazioni essenziali anche a fronte di una significativa riduzione dei dati. Viene presentata l'implementazione della compressione su immagini a colori (RGB) sfruttando la classe “ImageCompressor” implementata nei capitoli precedenti ed arricchendola con nuove funzionalità. Particolare attenzione viene posta all'elaborazione indipendente dei canali e alla sogliatura dei coefficienti wavelet. I risultati sperimentali mostrano una buona qualità visiva dell'immagine compressa, rendendo questa tecnica particolarmente adatta a scenari di archiviazione e trasmissione di dati visivi.

4.1 Compressione di un singolo canale

```
1 def __compress_wavelet_channel(  
2     self,  
3     channel,  
4     wavelet="haar",  
5     level=1,  
6     threshold=20,  
7     keep_ll_only=True,  
8     save_coeffs=False,  
9     channel_name="",  
10    output_dir="wavelet_coeffs"  
11 )  
12
```

Figura 4.1: Definizione della funzione `__compress_wavelet_channel`.

La funzione privata `__compress_wavelet_channel`, definita in Figura 4.1, viene introdotta nella classe `ImageCompressor` definita nei capitoli precedenti in Figura 2.1. Tale funzione rappresenta il passaggio fondamentale di applicazione della **trasformata wavelet discreta** a un singolo canale dell'immagine trattata in input.

I principali parametri passati alla funzione sono:

- **channel**: il canale dell'immagine (Rosso, Verde o Blu) su cui applicare la compressione.
- **wavelet**: la tipologia di wavelet madre da utilizzare per la trasformata discreta (es. `haar`, `db2`, `bior1.3`, ecc.).
- **level**: livello di decomposizione wavelet da applicare. Livelli più profondi consentono una maggiore astrazione dell'immagine, riducendo progressivamente i dettagli ad alta frequenza. Aumentando questo parametro si può ottenere una compressione più aggressiva.
- **threshold**: valore numerico utilizzato per la sogliatura dei coefficienti wavelet. Valori più alti comportano una maggiore eliminazione delle informazioni di dettaglio, rendendo la compressione più visibile ma più distruttiva.

```
1 coeffs = pywt.wavedec2(channel, wavelet=wavelet, level=level)
2 coeffs_compressed = []
3
```

Figura 4.2: Chiamata alla funzione `wavedec2`.

- **keep_ll_only**: valore booleano. Se `True`, conserva soltanto la componente LL (approssimazione) e azzerà tutte le componenti di dettaglio (LH, HL, HH). Se `False`, vengono mantenute (e sogliate) anche le componenti di dettaglio.

Tale funzione opera quindi applicando la DWT al canale in input, effettuando la sogliatura, e infine ricostruendo il canale compresso sulla base delle componenti desiderate (LL o tutte). I risultati ottenuti variano sensibilmente al variare dei parametri, in particolare del tipo di wavelet e del valore della soglia.

Utilizzo della funzione `wavedec2`

La funzione `wavedec2`, fornita dalla libreria `PyWavelets` (alias `pywt`), consente di applicare la trasformata wavelet discreta bidimensionale multilivello a una matrice, come nel caso di un singolo canale dell'immagine. Il parametro `level` controlla la profondità della decomposizione: un valore maggiore permette di estrarre informazioni a scale sempre più grandi, con una riduzione progressiva della risoluzione spaziale.

Come mostrato nella Figura 4.2, la chiamata restituisce una decomposizione gerarchica in cui il primo elemento è la matrice LL e i successivi sono tuple di coefficienti di dettaglio (LH, HL, HH) per ciascun livello. Questi coefficienti rappresentano la scomposizione spaziale e frequenziale del segnale, e sono così suddivisi:

- **LL** (Low-Low): rappresenta un'approssimazione a bassa frequenza dell'immagine originale. È la componente più significativa e contiene la maggior parte dell'informazione visiva utile.
- **LH** (Low-High): evidenzia le variazioni verticali ad alta frequenza (es. contorni orizzontali).

- **HL** (High-Low): cattura le variazioni orizzontali ad alta frequenza (es. contorni verticali).
- **HH** (High-High): rappresenta le variazioni diagonali ad alta frequenza (es. dettagli fini o rumore).

Questi coefficienti sono fondamentali per la compressione: mantenere solo i coefficienti LL consente una riduzione significativa della dimensione dell'immagine, sebbene a discapito della qualità. In alternativa, è possibile mantenere anche le componenti ad alta frequenza (LH, HL, HH) e applicare su di esse una sogliatura per eliminare dettagli ritenuti trascurabili, come discusso nella sezione successiva.

L'utilizzo di `wavedec2` permette dunque una decomposizione spaziale dell'immagine su più scale (grazie al parametro `level`), rendendola adatta a tecniche di compressione scalabili e controllabili in base al livello di qualità e dettaglio desiderato.

Cosa avviene dietro le quinte di `wavedec2`

Internamente, la trasformata `wavedec2` applica la decomposizione 1D wavelet separatamente su righe e colonne dell'immagine. Il procedimento può essere descritto come segue:

1. **Filtro orizzontale:** ogni riga della matrice viene convoluta con due filtri: uno passa-basso (low-pass filter) e uno passa-alto (high-pass filter). Questo produce due nuove matrici temporanee: una contenente le componenti a bassa frequenza e una con quelle ad alta frequenza lungo l'asse orizzontale.
2. **Sottocampionamento:** dopo la convoluzione, le matrici risultanti vengono sottocampionate (cioè ridotte di dimensione) mantenendo un elemento ogni due. Questo processo riduce la risoluzione spaziale della rappresentazione.
3. **Filtro verticale:** le stesse operazioni di convoluzione e sottocampionamento vengono poi applicate alle colonne delle due matrici ottenute al passo precedente.
4. **Combinazione:** alla fine si ottengono quattro matrici di dimensioni ridotte, corrispondenti rispettivamente ai coefficienti LL, LH, HL, HH.

```
1 for i, coeff in enumerate(coeffs):
2     if i == 0:
3         LL = pywt.threshold(coeff, threshold, mode='soft')
4         coeffs_compressed.append(LL)
5     else:
6         LH, HL, HH = coeff
7         if keep_ll_only:
8             coeffs_compressed.append((
9                 np.zeros_like(LH),
10                np.zeros_like(HL),
11                np.zeros_like(HH)
12            ))
13        else:
14            LH = pywt.threshold(LH, threshold, mode='soft')
15            HL = pywt.threshold(HL, threshold, mode='soft')
16            HH = pywt.threshold(HH, threshold, mode='soft')
17            coeffs_compressed.append((LH, HL, HH))
18
```

Figura 4.3: Applicazione della soglia `threshold` a tutti i coefficienti wavelet. Anche la componente LL viene sogliata, mentre le componenti di dettaglio possono essere azzerate o sogliate in base al parametro `keep_ll_only`.

Questo schema di filtraggio-separabile (prima righe, poi colonne) è alla base di molte trasformate bidimensionali ed è particolarmente efficiente sia in termini computazionali sia in termini di organizzazione gerarchica dell'informazione.

Applicazione della soglia ai coefficienti

In seguito alla decomposizione wavelet, i coefficienti ottenuti vengono sogliati al fine di ridurre la quantità di dati da conservare. Come mostrato in Figura 4.3, il processo di sogliatura viene applicato a tutti i coefficienti, compresa la componente di approssimazione LL, al fine di ridurre il più possibile la presenza di coefficienti bassi.

L'intero processo avviene iterando sulla lista di coefficienti `coeffs` restituita dalla funzione `wavedec2`. Per ciascun coefficiente:

- **Componente LL ($i = 0$):** viene sogliata direttamente utilizzando il metodo `pywt.threshold`, con modalità “soft”, per rimuovere le componenti a bassa intensità anche nell’approssimazione globale.
- **Componenti LH, HL, HH ($i > 0$):** se il flag `keep_ll_only` è attivo, tutte le componenti di dettaglio vengono azzerate mediante matrici di zeri; in caso contrario, ciascuna di esse viene sogliata individualmente sempre in modalità “soft”.

La sogliatura in modalità `soft` riduce gradualmente i coefficienti in prossimità della soglia, evitando brusche transizioni e migliorando la qualità della ricostruzione. L’inclusione della sogliatura sulla componente LL rappresenta una scelta progettuale mirata a ottenere una compressione più aggressiva, potenzialmente a discapito della qualità complessiva, ma efficace in contesti dove è prioritario ridurre l’informazione ridondante a tutti i livelli.

Il parametro `keep_ll_only` regola, invece, il comportamento sui coefficienti di dettaglio:

- Se impostato a `True`, tutti i dettagli (LH, HL, HH) vengono azzerati, lasciando come unica informazione residua la componente LL sogliata.
- Se impostato a `False`, anche i dettagli vengono sogliati, conservando una parte delle informazioni ad alta frequenza.

Questo approccio consente una maggiore flessibilità nella regolazione del grado di compressione, permettendo di bilanciare qualità e dimensione in modo più fine.

Ricostruzione del canale compresso

Dopo l’applicazione della trasformata wavelet discreta, la sogliatura dei coefficienti e l’eventuale eliminazione delle componenti ad alta frequenza, è necessario ricostruire il canale dell’immagine a partire dai coefficienti modificati. Questa operazione viene eseguita tramite la trasformata wavelet inversa bidimensionale, resa disponibile dalla funzione `waverec2` della libreria `PyWavelets`.

Come mostrato in Figura 4.4, la funzione `waverec2` prende in input la lista di coefficienti modificati (`coeffs_compressed`), che include la componente LL e tutti


```
1 reconstructed = pywt.waverec2(coeffs_compressed, wavelet=wavelet)
2 return np.clip(reconstructed, 0, 255)
3
```

Figura 4.4: Il canale compresso viene ricostruito per essere utilizzato nella funzione di compressione totale. Per la ricostruzione viene sfruttata la trasformata inversa `waverec2`.

```
1 def compress_color_image_dwt(
2     self,
3     wavelet="haar",
4     level=1,
5     threshold=20,
6     keep_ll_only=True,
7     ...
8 )
9
```

Figura 4.5: Definizione della funzione `compress_color_image_dwt`.

i dettagli (LH, HL, HH) a ciascun livello. Il risultato è una matrice bidimensionale che rappresenta il canale ricostruito.

Durante il processo di trasformazione e ricostruzione, alcuni valori dei pixel possono uscire dal range standard `[0, 255]`, specialmente in presenza di sogliature aggressive o perdita di informazione. Per questo motivo, il risultato della `waverec2` viene passato alla funzione `np.clip`, che forza tutti i valori a rientrare nel dominio valido per immagini in scala di grigi o canali RGB a 8 bit.

Questo passaggio finale consente di integrare il canale ricostruito all'interno dell'immagine compressa, garantendo compatibilità con i formati e le librerie grafiche comuni.

4.2 Compressione dell'immagine RGB

Il processo di conversione completo dell'immagine viene definito nella funzione `compress_color_image` definita in Figura 4.5. Anche in questo caso, il funzio-

namento è basato sul processo ciclico di chiamata alla funzione di compressione del singolo canale, nello specifico, `__compress_wavelet_channel`. Lo script viene impostato per immagini RGB ma è facilmente rimodellabile per immagini che sfruttano altri tipi di canali.

Fase preliminare

Le prime operazioni svolte nella funzione di compressione sono quelle già riportate nei capitoli precedenti in Figura 2.9. In particolare, si converte l'immagine in input in formato RGB per assicurare la presenza dei tre canali su cui lavorare (Rosso, Verde, Blu) per poi tradurre l'immagine risultante in un array NumPy tridimensionale. Questa serie di operazioni permette di elaborare l'immagine in maniera standardizzata e, in particolare, sui valori contenuti in ciascun canale separatamente.

Ciclo di chiamata alla funzione di compressione del canale

Come mostrato nella Figura 4.6, una volta convertita l'immagine in un array NumPy tridimensionale, si procede all'elaborazione separata dei tre canali RGB. Questo approccio modulare consente di applicare la compressione wavelet in modo indipendente su ciascun componente cromatico, permettendo una gestione più fine e flessibile dei dati visivi.

Il ciclo `for` itera sui tre canali: Rosso, Verde e Blu. Per ciascun canale, viene estratta la relativa matrice bidimensionale di intensità, rappresentante i livelli di colore per ogni pixel dell'immagine. Questa matrice viene quindi passata alla funzione privata `__compress_wavelet_channel`, la quale si occupa dell'intera catena di compressione: trasformata wavelet discreta (`wavedec2`), sogliatura dei coefficienti, eventuale annullamento delle componenti ad alta frequenza e ricostruzione del canale tramite la trasformata inversa (`waverec2`).

Una volta ottenuto il canale compresso, si procede ad effettuare un'operazione di conversione esplicita in tipo `uint8` per garantire compatibilità con le librerie di visualizzazione e salvataggio di immagini.

Infine, il canale compresso viene aggiunto alla lista `compressed_channels`. Al termine dell'iterazione, questa lista conterrà i tre canali RGB compressi separatamente e sarà pronta per la fase di ricostruzione dell'immagine finale. Questo ciclo

```
1 compressed_channels = []
2 channel_names = ['Rosso', 'Verde', 'Blu']
3
4 for i in range(3):
5     channel = img_np[:, :, i]
6     compressed = self.__compress_wavelet_channel(
7         channel=channel,
8         wavelet=wavelet,
9         threshold=threshold,
10        keep_ll_only=keep_ll_only,
11        ...
12    )
13    compressed_channels.append(compressed.astype(np.uint8))
14
```

Figura 4.6: Iterazione sui tre canali dell'immagine RGB con corrispondente chiamata alla funzione `__compress_wavelet_channel` per ciascuno. Ogni canale restituito viene convertito in valori interi ad 8 bit e aggiunto alla lista dei canali compressi.

rappresenta un punto cruciale dell'intero processo, poiché assicura che la trasformata wavelet venga applicata in modo coerente e controllato a tutte le componenti dell'immagine, mantenendo un buon equilibrio tra compressione e qualità visiva.

Ricostruzione dell'immagine compressa

Una volta completato il processo di compressione sui tre canali RGB in maniera indipendente, si procede con la fase di ricostruzione dell'immagine finale. Come mostrato nei capitoli precedenti, in particolare nella Figura 2.11, questa operazione prevede la combinazione dei tre canali compressi in un'unica struttura tridimensionale tramite la funzione `np.stack`, specificando l'asse 2 per indicare la dimensione del canale colore. Il risultato è una matrice tridimensionale che rappresenta l'immagine compressa nel formato RGB.

L'operazione di **stacking** ripristina la struttura originaria dell'immagine a tre canali, ma con informazioni che ora risultano compresse in base ai parametri scelti (tipo di wavelet, soglia, mantenimento o meno dei coefficienti ad alta frequenza).



Figura 4.7: Immagine originale del cubo di Rubik.

Infine, la funzione `Image.fromarray` della libreria `PIL` converte la matrice NumPy risultante in un oggetto immagine compatibile con i formati grafici standard, permettendo così la visualizzazione o il salvataggio su disco del risultato finale. Questo passaggio segna il termine del processo di compressione, restituendo un'immagine visivamente simile all'originale ma di dimensione potenzialmente ridotta.

4.3 Esempio: Cubo di Rubik

Il processo di compressione tramite wavelet è particolarmente modellabile in base alle necessità. Si è deciso in tal senso di sperimentare diverse combinazioni di parametri sull'immagine del cubo di Rubik analizzata in precedenza e riportata in Figura 4.7. Si noti che per ogni “esperimento” viene riportata la chiamata a funzione che lo ha generato nella quale vengono specificati i dettagli della compressione.

```
1 rubik.compress_color_image_dwt(  
2     wavelet="db2",  
3     level=2,  
4     threshold=50,  
5     keep_ll_only=False,  
6     show=True,  
7     save_name="compressed_light.jpg",  
8     save_coeffs=True  
9 )  
10
```

Figura 4.8: Compressione leggera effettuata utilizzando la wavelet madre “Daubechies” db2, con livello di decomposizione pari a 2, soglia (threshold) pari a 50 e mantenimento di tutte le componenti wavelet (keep_ll_only=False).

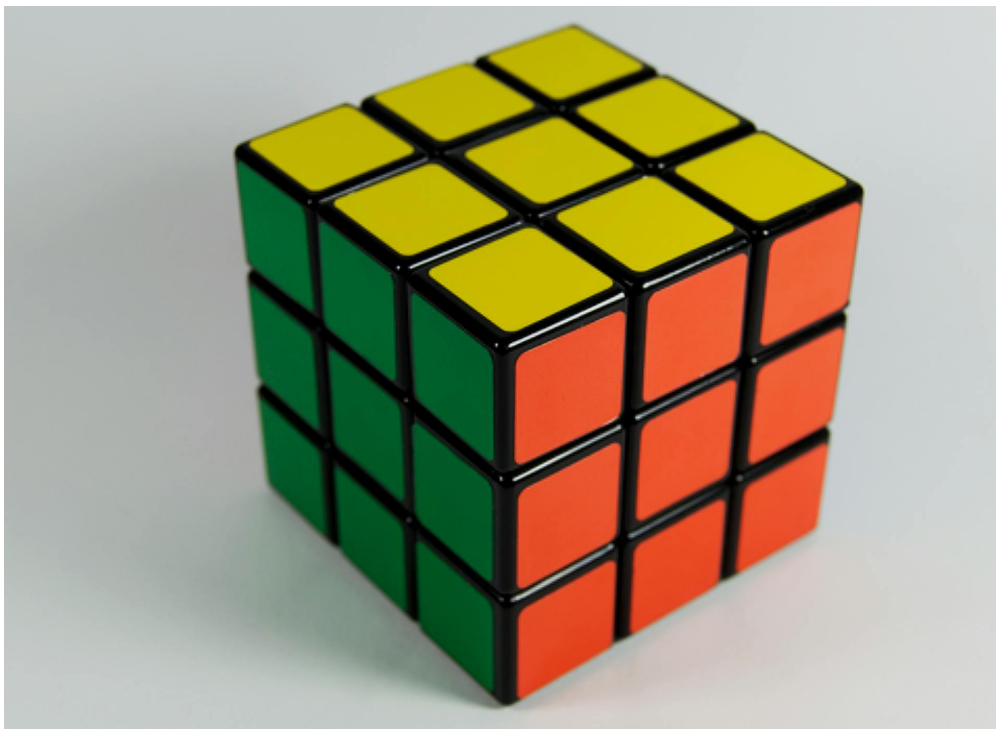


Figura 4.9: Immagine compressa utilizzando il codice in Figura 4.8. Il risultato è visivamente molto simile all'originale, con perdita di dettaglio trascurabile.

```
1 rubik.compress_color_image_dwt(  
2     wavelet="haar",  
3     level=3,  
4     threshold=80,  
5     keep_ll_only=False,  
6     show=True,  
7     save_name="compressed_medium.jpg",  
8     save_coefs=True  
9 )  
10
```

Figura 4.10: Compressione di livello medio effettuata utilizzando la wavelet madre “Haar”, con livello di decomposizione pari a 3, soglia (`threshold`) pari a 80 e mantenimento di tutte le componenti wavelet (`keep_ll_only=False`).



Figura 4.11: Immagine compressa utilizzando il codice in Figura 4.10. La qualità visiva risulta ancora buona, sebbene con una leggera perdita di dettagli fini.

```
1 rubik.compress_color_image_dwt(  
2     wavelet="db2",  
3     level=3,  
4     threshold=100,  
5     keep_ll_only=True,  
6     show=True,  
7     save_name="compressed_LLonly.jpg",  
8     save_coeffs=True  
9 )  
10
```

Figura 4.12: Compressione forte effettuata utilizzando la wavelet madre “Daubechies” db2, con livello di decomposizione pari a 3, soglia (threshold) pari a 100 e mantenimento esclusivo della componente LL (keep_ll_only=True).



Figura 4.13: Immagine compressa utilizzando il codice in Figura 4.12. La rimozione completa dei dettagli comporta una notevole perdita di nitidezza, ma consente un'alta compressione.

```
1 rubik.compress_color_image_dwt(  
2     wavelet="haar",  
3     level=4,  
4     threshold=150,  
5     keep_ll_only=True,  
6     show=True,  
7     save_name="compressed_aggressive.jpg",  
8     save_coeffs=True  
9 )  
10
```

Figura 4.14: Compressione aggressiva effettuata utilizzando la wavelet madre “Haar”, con livello di decomposizione pari a 4, soglia (`threshold`) pari a 150 e mantenimento esclusivo della componente LL (`keep_ll_only=True`).

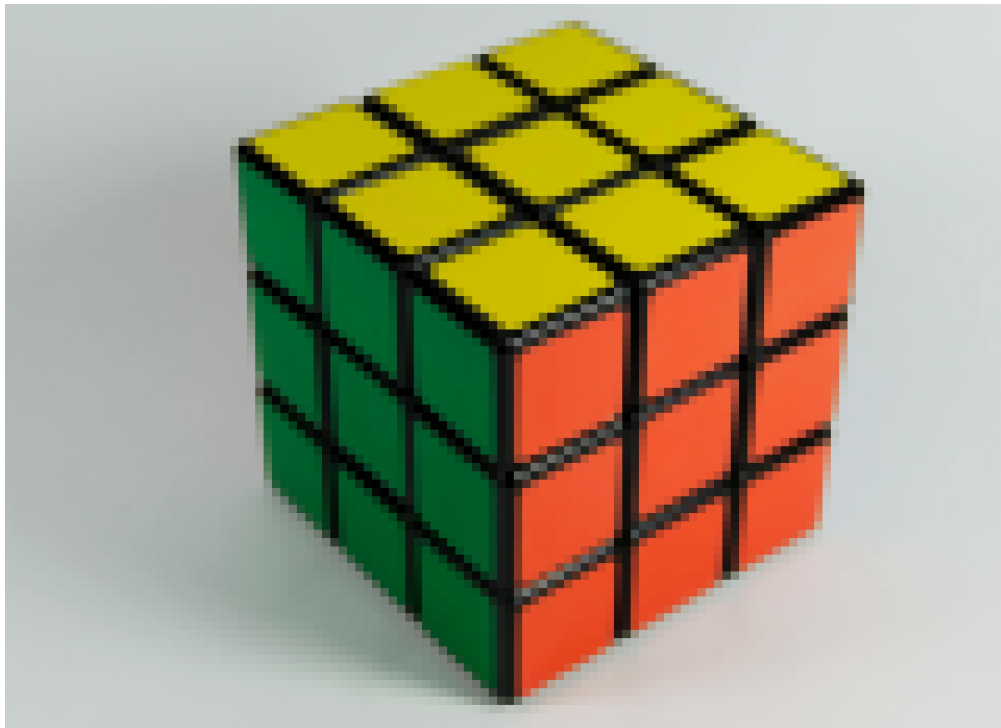


Figura 4.15: Immagine compressa utilizzando il codice in Figura 4.14. L'effetto risultante è una forte sfocatura dell'immagine, con conservazione minima delle informazioni visive, a favore di un'elevata riduzione dei dati.